

Survey of Simulators for Aerial Robots

Cora A. Dimmig^{1,2}, Giuseppe Silano³, Kimberly McGuire⁴, Chiara Gabellieri⁵, Wolfgang Hönig⁶, Joseph Moore^{1,2}, and Marin Kobilarov¹

Abstract—Uncrewed Aerial Vehicle (UAV) research faces challenges with safety, scalability, costs, and ecological impact when conducting hardware testing. High-fidelity simulators offer a vital solution by replicating real-world conditions to enable the development and evaluation of novel perception and control algorithms. However, the large number of available simulators poses a significant challenge for researchers to determine which simulator best suits their specific use-case, based on each simulator’s limitations and customization readiness. In this paper we present an overview of 43 UAV simulators, including in-depth, systematic comparisons for 17 of the simulators. Additionally, we present a set of decision factors for selection of simulators, aiming to enhance the efficiency and safety of research endeavors.

I. INTRODUCTION

Uncrewed Aerial Vehicles (UAVs) are being widely adopted for a variety of use cases and industries, such as agriculture, inspection, mapping, and search and rescue [1]. In particular, aerial manipulation and human-robot interaction applications have been on the rise, including tasks such as parcel delivery, sample collection, and collaborative robot operations [2], [3].

Testing experimental algorithms directly on hardware can pose significant risks, as unexpected behaviors may emerge. Moreover, crashes can incur substantial costs, disrupt development schedules, and contribute to environmental harm due to the frequent replacement of damaged vehicle components. Additionally, in the context of the increasing adoption of machine learning-based techniques, collecting data from hardware can prove highly inefficient and often impracticable. Hence, a dependable, fast, precise, and realistic UAV simulator is essential to facilitate rapid advancements in this field. Due to the rise of high-fidelity simulators, results from simulation can often be efficiently transferred to hardware, however challenges may arise in domains with unmodeled effects (e.g. agile flight, close-proximity flight, UAVs with manipulators).

In this work, we analyze some of the prominent UAV simulators and discuss key selection criteria and decision factors to consider when choosing a simulator. To illustrate the breadth of existing simulators, we include simulators that

are widely used (as high adoption rates often result in more testing and community input) and simulators specialized for popular research areas. Figure 1 demonstrates the large spectrum of simulators and their use cases. This research builds upon discussions and contributions made during the workshop titled “*The Role of Robotics Simulators for Unmanned Aerial Vehicles*” at the 2023 International Conference on Robotics and Automation (ICRA) in London, UK¹. Specifically, our work addresses the following question:

Question: What criteria and considerations should guide the selection and customization of a simulator to optimize its suitability for a specific application, while also understanding its limitations?

There are a few existing survey papers focusing on simulators and their role in robotics [4]. For example, a recent survey [5] analyzes a wide range of application areas, including aerial vehicles, and compares features across diverse domains. Regrettably, not every simulator readily supports UAVs. Dynamics considerations for manipulators or ground vehicles can substantially differ from those of aerial vehicles, especially in research that aims to account for aerodynamic effects. In [6], the authors examine various considerations for aerial delivery vehicles, including simulator selection. In [7], the authors analyze simulators specific to aerial vehicles, including some less commonly used simulators, and discuss criteria for simulator selection.

We believe that the ensuing discussion and difficulty in selecting a simulator requires a more focused survey paper covering an expanded list of UAV simulators. We provide readers with valuable insights based on our experiences in international robotics competitions, innovative research projects, and real-world applications, contemplate the future of simulation tools, and provide consolidated information for readers to explore effective solutions for their intended applications.

UAV DYNAMICS BACKGROUND

This section delves into fundamental concepts crucial for a comprehensive understanding of UAV dynamics. We focus on UAVs without morphing capabilities (i.e. the ability for a vehicle to change its shape). We express variables and parameters in Table I and vehicle schematics in Figure 2.

A. Multirotors

Basic: The multirotor’s dynamics are modeled as a 6-Degree-of-Freedom (DoF) floating rigid body using Newton-Euler formalism with squared motor speed inputs [1]. The motors exert forces and torques on the Center of Mass (CoM).

¹Department of Mechanical Engineering, Johns Hopkins University, Baltimore, Maryland, USA (emails: {cdimmig, marin}@jhu.edu).

²Johns Hopkins University Applied Physics Laboratory, Laurel, Maryland, USA (email: joseph.moore@jhuapl.edu).

³Department of Cybernetics, Czech Technical University in Prague, Prague, Czech Republic (email: giuseppe.silano@fel.cvut.cz).

⁴Bitcraze A.B., Malmö, Sweden (email: kimberly@bitcraze.io).

⁵Robotics and Mechatronics (RaM) Group, University of Twente, Enschede, The Netherlands (email: c.gabellieri@utwente.nl).

⁶Intelligent Multi-Robot Coordination Lab, Technische Universität (TU) Berlin, Germany (email: hoenig@tu-berlin.de).

This work was partially funded by the National Science Foundation grant no. 1925189, by the EU’s MSCA FLYFLIC grant no. 101059875, by the EU’s H2020 AERIAL-CORE grant no. 871479, and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - 448549715.

¹<https://imrclab.github.io/workshop-uav-sims-icra2023>

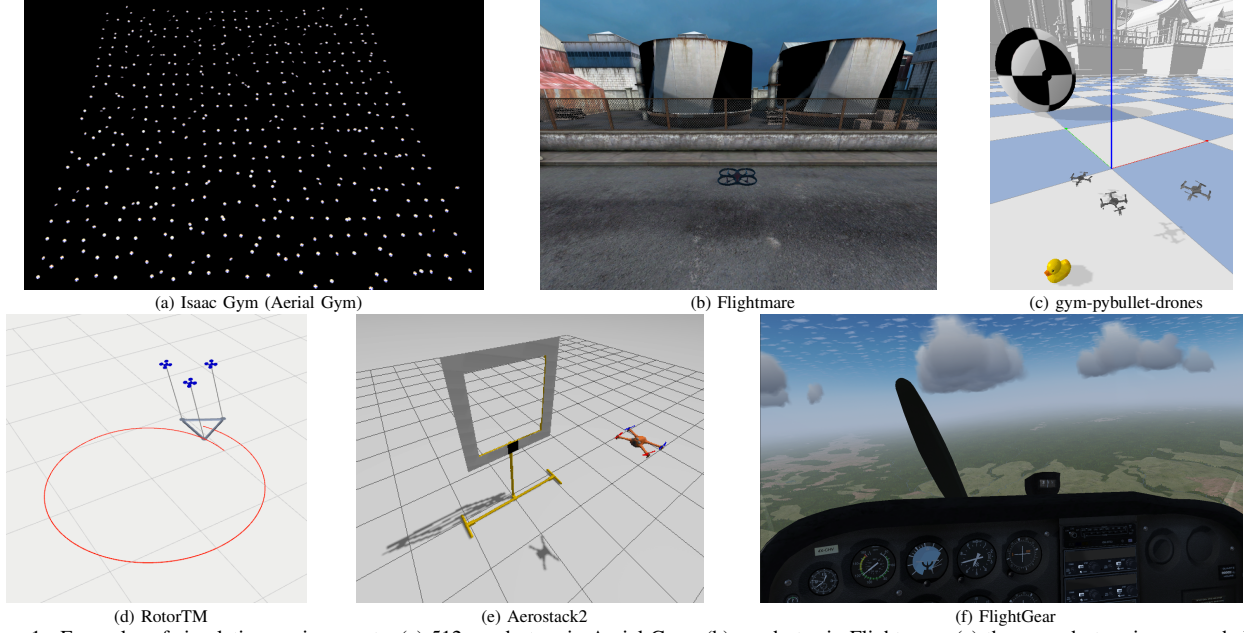


Figure 1. Examples of simulation environments: (a) 512 quadrotors in Aerial Gym, (b) quadrotor in Flightmare, (c) three quadrotors in gym-pybullet-drones, (d) three quadrotors with a triangular payload in RotorTM, (e) quadrotor with a racing gate in Aerostack2, and (f) Cessna 172P cockpit view in FlightGear.

$$\begin{aligned}
 \text{State: } \mathbf{x} &= (\mathbf{p}, \mathbf{q}, \mathbf{v}, \boldsymbol{\omega})^\top, \\
 \text{Control Inputs: } \mathbf{u}_\Omega &= (\Omega_1, \dots, \Omega_n)^\top, \\
 \text{Forces: } \mathbf{f} &= \sum_i c_{f_i} \Omega_i \mathbf{z}_{\Omega_i} = \mathbf{F} \mathbf{u}_\Omega, \\
 \text{Torques: } \boldsymbol{\tau} &= \sum_i (c_{\tau_i} \mathbf{p}_{\Omega_i} \times \mathbf{z}_{\Omega_i} + c_{\tau_i} \mathbf{z}_{\Omega_i}) \Omega_i = \mathbf{M} \mathbf{u}_\Omega, \\
 \text{Dynamics:} \\
 \dot{\mathbf{p}} &= \mathbf{v}, & m \dot{\mathbf{v}} &= m \mathbf{g} + \mathbf{R}(\mathbf{q}) \mathbf{F} \mathbf{u}_\Omega + \mathbf{f}_a, \\
 \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \circ \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}, & \mathbf{J} \dot{\boldsymbol{\omega}} &= -\boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega} + \mathbf{M} \mathbf{u}_\Omega + \boldsymbol{\tau}_a, \quad (1)
 \end{aligned}$$

where $\circ$ and $\times$ represent quaternion and vector cross products, respectively. The model can be extended for UAVs with tilting propellers by introducing a control variable $\mathbf{u}_w$ for real-time adjustment of $\mathbf{F}(\mathbf{u}_w)$ and $\mathbf{M}(\mathbf{u}_w)$ matrices [2].

Drag: At high speeds, multirotors experience aerodynamic drag forces and torques, often considered disturbances proportional to the velocity ($\mathbf{f}_a \propto \mathbf{v}$ and $\boldsymbol{\tau}_a \propto \boldsymbol{\omega}$) [8].

Wind: Wind is typically modeled by a spatio-temporal-varying external force $\mathbf{f}_a(\mathbf{p}_w, t)$, where $\mathbf{p}_w$ and t represent position and time, respectively [1].

Interactions: In close-proximity flight, multirotors experience aerodynamic interaction forces, often modeled as a learned function $\mathbf{f}_a$ based on relative neighbor poses [9].

B. Helicopters

Basic: Helicopters have similar dynamics to multirotors, mainly differing in force $\mathbf{F}$ and torque $\mathbf{M}$ allocation matrices, which are two-dimensional and potentially not fixed [1].

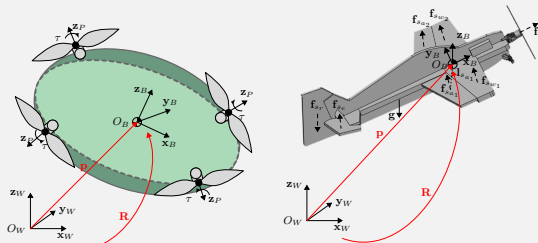


Figure 2. Schematic representations of a multirotor and fixed-wing with global frames O_W and body frames O_B .

Table I
UAV DYNAMICS VARIABLES

Var.	Set	Description
m	$\mathbb{R}$	Mass
$\mathbf{J}$	$\mathbb{R}^{3 \times 3}$	Inertia
$\mathbf{p}$	$\mathbb{R}^3$	Position
$\mathbf{q}$	$\mathbb{R}^4$	Attitude unit quaternion
$\mathbf{v}$	$\mathbb{R}^3$	Global velocity
$\boldsymbol{\omega}$	$\mathbb{R}^3$	Body angular velocity
Ω_i	$\mathbb{R}_{\geq 0}$	i -th squared motor speed
c_{f_i}	$\mathbb{R}$	Propeller's shape force parameter
c_{τ_i}	$\mathbb{R}$	Propeller's shape torque parameter
$\mathbf{z}_{\Omega_i}$	$\mathbb{R}^3$	Unit vector parallel to the rotor's rotation axis
$\mathbf{p}_{\Omega_i}$	$\mathbb{R}^3$	Rotor's position in the body frame
$\mathbf{F}$	$\mathbb{R}^{3 \times n}$	Force allocation matrix
$\mathbf{M}$	$\mathbb{R}^{3 \times n}$	Torque allocation matrix
g	$\mathbb{R}$	Acceleration due to gravity, $\mathbf{g} = (0, 0, -g)^\top$
$\mathbf{R}$	$SO(3)$	Body-to-global rotation matrix
$\mathbf{f}_a$	$\mathbb{R}^3$	External forces acting on the UAV
$\boldsymbol{\tau}_a$	$\mathbb{R}^3$	External torques acting on the UAV
$\mathbf{f}_{s_i}$	$\mathbb{R}^3$	External force from aerodynamic surface s_i
$\mathbf{l}_{s_i}$	$\mathbb{R}^3$	Displacement from CoM to center of lift
ρ	$\mathbb{R}$	Air density
$\mathbf{v}_{s_i}$	$\mathbb{R}^3$	Relative wind at aerodynamic center in s_i frame
S_i	$\mathbb{R}$	Surface area of the i -th surface
c_{L_i}	$\mathbb{R}$	Lift coefficient for the i -th surface
c_{D_i}	$\mathbb{R}$	Drag coefficient for the i -th surface
$\mathbf{e}_{D_i}$	$\mathbb{R}^3$	Unit vector in the drag direction ($\mathbf{v}_{s_i} / \ \mathbf{v}_{s_i}\ $)
$\mathbf{e}_{L_i}$	$\mathbb{R}^3$	Unit vector in the lift direction ($\perp$ to $\mathbf{e}_{D_i}$)
$\mathbf{R}_{s_i}$	$SO(3)$	Body-to-surface rotation matrix
$\mathbf{f}_e$	$\mathbb{R}^3$	Body frame environment force on end-effector
$\boldsymbol{\tau}_e$	$\mathbb{R}^3$	Body frame environment torque on end-effector
$\mathbf{p}_e$	$\mathbb{R}^3$	End-effector tip's position in body frame

C. Fixed-wings

Basic: For fixed-wing UAVs, like multirotors, equations of motion derive from Newton-Euler formalism. We extend (1) with the following external aerodynamic forces and torques.

External Aerodynamic Forces: $\mathbf{f}_a = \sum_i \mathbf{f}_{s_i}$,

External Aerodynamic Torques: $\boldsymbol{\tau}_a = \sum_i (\mathbf{l}_{s_i} \times \mathbf{f}_{s_i})$,

Surface Forces: $\mathbf{f}_{s_i} = \frac{1}{2} \rho |\mathbf{v}_{s_i}|^2 S_i (c_{L_i} \mathbf{e}_{L_i} + c_{D_i} \mathbf{e}_{D_i})$,

Relative Wind: $\mathbf{v}_{s_i} = -\mathbf{R}_{s_i}^\top (\mathbf{R}(\mathbf{q})^\top \mathbf{v} + \boldsymbol{\omega} \times \mathbf{l}_{s_i})$.

For control surfaces with a single DoF, both $\mathbf{R}_{s_i}$ and $\mathbf{l}_{s_i}$ depend on the control surface deflection, δ_i , which is treated as additional control variables, denoted by $\mathbf{u}_\delta = (\delta_1, \dots, \delta_c)^\top$. Typical aerodynamic surfaces include wings and vertical or horizontal stabilizers; typical control surfaces are ailerons, rudder, and elevator [10], [11].

Lift and Drag: For fixed-wings, lift and drag forces play pivotal roles in vehicle simulation. The coefficients c_{L_i} and c_{D_i} can be derived by combining airfoil data at low angle-of-attack with high angle-of-attack approximations obtained from flat-plate theory [12]. Additionally, these coefficients can also be partly or entirely data-driven [13].

Wind: Wind is modeled as a spatio-temporal-varying additive velocity contributing directly to $\mathbf{v}_{s_i}$ [14].

Interactions: Ground effects significantly impact fixed-wing UAVs during landing maneuvers. Data-driven approaches effectively capture this effect by relating c_{L_i} to ground proximity [15].

D. Aerial manipulators

Basic: Aerial manipulators consist of an aerial base and an end-effector (e.g. rigid tools, articulated arms, cables). Here, we include a general description, excluding advanced techniques like soft manipulation [16].

Rigid Tools: Aerial manipulators with rigid tools follow (1) with adjustments for the tool's inertial properties. The terms $\mathbf{f}_a$ and $\boldsymbol{\tau}_a$ account for the wrench applied at the end-effector.

External Force: $\mathbf{f}_a = \mathbf{R}(\mathbf{q})\mathbf{f}_e$,

External Torque: $\boldsymbol{\tau}_a = \mathbf{p}_e \times \mathbf{f}_e + \boldsymbol{\tau}_e$.

Articulated Arms: For manipulators with articulated arms, dynamics become more complex [17].

Cables: Cables are typically modeled as massless rigid or elastic elements attached via passive spherical joints to the robot's CoM [18]. In this scenario, $\mathbf{f}_a = \mathbf{R}(\mathbf{q})\mathbf{f}_e$ accounts for the cable's force. In this formulation, external torque applied to the robot by the cable is excluded ($\boldsymbol{\tau}_a = \mathbf{0}_{3 \times 1}$) [17].

II. UAV SIMULATORS

A primary consideration when selecting a simulator is the specific application domain and whether the available simulators offer the necessary features and sensors tailored to that domain. Additionally, compatibility with common autopilots (e.g. PX4 and ArduPilot) or standardized research hardware (e.g. the Crazyflie, a palm sized UAV) is often a consideration to enable rapid simulation to hardware transfer. Many specialized simulators have emerged tailored to these components. Drawing from our own experiences and the referenced literature, we compiled a set of selection criteria and decision factors that are regularly considered when evaluating UAV simulators. These comparative points are outlined in Table II. Table III categorizes a range of UAV simulators based on their key elements. We consider the following categories: a “Uni-

Table II
SELECTION CRITERIA FOR UAV SIMULATORS

Criteria	Decision Factors
Physics Fidelity	Required fidelity of physics and dynamics model for the intended use case
Visual Fidelity	Necessity for realistic images (e.g. for computer vision or Machine Learning (ML) applications)
Autopilots	Compatibility with common autopilots like PX4 and ArduPilot, useful for Software-In-The-Loop (SITL) and Hardware-In-The-Loop (HITL) testing
Multiple Vehicles	Capability to concurrently simulate vehicles
Heterogeneity	Integration possibilities with other platforms
Sensors	Integration support for common sensors (e.g. cameras, IMU, GPS, LiDAR, optical flow)
UAV Models	Support of common UAV models and ease of integrating new models
Simulation Speed	Real-time speed and ability to run in super real-time, crucial for learning applications
APIs	Compatibility with programming languages, middleware like ROS, and packages such as OpenAI Gym (now Gymnasium)
Integration	Ease of getting started and development, license type, and maintenance status of the software

versal Simulator” is targeted toward simulating (potentially simultaneously) a wide variety of platforms (e.g. ground and aerial vehicles, manipulators, etc.); “Sensor-Focused” targets simulating realistic performance of a particular sensor (e.g. for photorealism or accurate LiDAR measurements); “Learning-Focused” targets compatibility with ML architectures (e.g. for training a new behavior or learning a dynamics model); “Dynamics-Focused” emphasizes thorough dynamics models for a particular class of systems (e.g. aerial manipulation, like RotorTM, HIL-airmanip, or fixed-wings, like PyFly); “Swarming” simulates a group of vehicles; “Part of Flight Stacks” are simulators with flight stack integration; and “Flight Simulators” are flight emulators often for large-scale human-piloted aircraft. Flight simulators are less often used for robotics applications, however some have been adapted for this purpose due to their realistic flight experience and we have included them here for completeness. Some simulators may belong to multiple categories.

What follows offers a concise overview of each simulator featured in Table III. We present each simulator in a subsection corresponding to one of its major categories, with references in the other categories corresponding to Table III.

A. Universal simulators

Gazebo Classic [19] is an open-source, continuously maintained, versatile research simulation platform with a modular design, accommodating different physics engines, sensors, and 3D world creation. Particularly noteworthy is its suitability for aerial manipulator tasks, owing to its ease of creating contact surfaces with customizable frictions [20], [21]. *RotorS* [22], built on top of Gazebo Classic, offers a modular framework for designing UAVs and developing control algorithms, particularly focusing on simulating the vehicle dynamics. *CrazyS* [23], an extension of RotorS, focuses on modeling the Crazyflie 2.0 quadrotor. However, both RotorS and CrazyS have limited perception-related capabilities. *PX4 SITL Gazebo*

Table III
UAV SIMULATORS CATEGORIZATION

Category	Simulators
Universal Simulators	Gazebo Classic (RotorS, CrazyS, PX4 SITL), Gazebo, Isaac Sim/Gym (Pegasus, Aerial Gym), Webots, CoppeliaSim, MuJoCo
Sensor-Focused	Vision: AirSim, Flightmare, FlightGoggles, FastSim LiDAR: MARSIM
Learning-Focused	Isaac Gym (Aerial Gym), MuJoCo, PRL4AirSim, Flightmare, gym-pybullet-drones, safe-control-gym, QuadSwarm, fixed-wing-gym, QPlane, PyFlyt
Dynamics-Focused	RotorTM, MATLAB UAV Toolbox, PyFly, ARCAD, HIL-airmanip, RotorPy, RflySim, Agilicious
Swarming	gym-pybullet-drones, QuadSwarm, Potato
Part of Flight Stacks	Agilicious, MRS UAV System, CrazyChoir, Crazyswarm2, Aerostack2, CrazySim, sim_cf2
Flight Simulators	X-Plane (X-PlaneROS), QPlane, FlightGear, RealFlight

[24], partially originated from RotorS, includes the latest support for PX4 SITL, does not depend on ROS, and supports simulating a large number of vehicles. Additionally, PX4 SITL includes an airspeed sensor, which is essential for fixed-wing and Vertical Take-Off and Landing (VTOL) vehicle simulations. The new *Gazebo* [25], formerly known as Ignition, is the successor of Gazebo Classic and incorporates quadrotor dynamics and control inspired by the RotorS project. Gazebo enables dynamic loading and unloading of environment components, addressing challenges faced by Gazebo Classic in replicating large, realistic environments. Moreover, Gazebo offers improved interfaces for simulating radio communication between multiple UAVs.

Isaac Sim [26], developed by NVIDIA, is a photorealistic high-fidelity simulator for a variety of platforms. *Pegasus Simulator* [27] is an open-source extension to Isaac Sim that includes an extended multirotor dynamics model, simulating multiple vehicles in parallel, integration with PX4 and ROS 2, and additional sensors (magnetometer, GPS, and barometer). *Isaac Gym* [28] is NVIDIA’s library for GPU-accelerated Reinforcement Learning (RL) simulations and uses more basic rendering than Isaac Sim. *Aerial Gym* [29] is an open-source extension to Isaac Gym Preview Release 4 notable for its capability to parallelize the simulation of thousands of multirotors and includes customizable obstacle randomization.

Webots [30] is an open-source, versatile robotics simulator known for its wide range of robotic platforms. While Webots primarily focuses on ground-bound robots, it also features two quadrotor models with simplified aerodynamic physics. Webots uses ODE for physics simulation, refer to [31] for a comprehensive analysis. Notably, Webots has been used for innovative vehicle designs, such as a triphibious robot in [32].

CoppeliaSim [33], previously known as V-REP, is a versatile robotics simulator with support for a wide range of programming languages and physics engines. Selecting the appropriate physics engine is crucial to avoid undesirable outcomes, such as velocity or position jumps, unrealistic collision behaviors, and erratic sensor outputs [31]. CoppeliaSim has been used for applications such as UAV obstacle avoidance [34].

MuJoCo [35], see Sec. II-C.

B. Sensor-focused simulators

AirSim [36] is a Microsoft-led project built on the Unreal Engine, offering various sensors, a weather API, and compatibility with open-source controllers like PX4. AirSim primarily serves as a platform for AI research, providing platform-independent APIs for data retrieval and vehicle control. Notably, AirSim demands substantial computing power compared to other simulators. *PRL4AirSim* [37] is an extension designed for efficient parallel training in RL applications. The original AirSim is open-source, but will no longer be supported by Microsoft. Their focus has shifted to *Project AirSim*, which will be released under a commercial license.

Flightmare [38] is a versatile simulator with two main components: a Unity-based rendering engine and a physics model, both designed for flexibility and independent operation. The rendering engine can generate realistic visual information and simulate sensor noise, environmental dynamics, and lens distortions with minimal computational overhead. Similarly, the physics model allows users to control robot dynamics, ranging from basic noise-free UAVs models to advanced rigid-body dynamics with friction and rotor drag, or even real platform dynamics. Flightmare is extensively used for ML applications, such as for autonomous drone racing [39].

FlightGoggles [40], similarly to Flightmare, is an open-source simulator focused on photorealistic simulation. FlightGoggles combines two key elements: (i) photogrammetry for realistic simulation of camera sensors, and (ii) virtual reality to integrate real vehicle motion and human behavior in the simulations. FlightGoggles is built around the Unity engine and includes multirotor physics with motor dynamics, basic vehicle aerodynamics, and IMU bias dynamics. A key feature of FlightGoggles is the “vehicle-in-the-loop simulation,” where the vehicle is flown in a motion capture system, camera images and exteroceptive sensors are simulated in Unity, and collision detection is based on the real-world vehicle’s pose.

FastSim [41] is a high-fidelity, modular simulation framework built on Unity (for photorealistic simulation). Configurable modules include sensors, algorithms, robots, worlds, and display tools. Implemented sensors include IMU, RGB, depth, segmentation, and event cameras. Both SITL and HITL are supported. Demonstrated applications include vision-based localization, motion planning, swarming, and ML.

MARSIM [42] is an open-source C/C++ library primarily focused on accurately simulating LiDAR measurements for UAVs. It constructs depth images from point cloud maps and interpolates them to obtain LiDAR point measurements. The simulator is designed for lightweight computation and offers access to 10 high-resolution environments, including forest, historic building, office, parking garage, and indoor settings.

C. Learning-focused simulators

MuJoCo [35], or Multi-Joint dynamics with Contact, is a frequently employed physics engine and simulator in ML applications. It offers interactive visualization rendered with OpenGL and encompasses various platforms, including a UAV model (i.e. the Skydio X2 quadrotor).

safe-control-gym [43] is an open-source safety-focused RL environment and benchmark suite, built using the PyBullet physics engine [44], for comparing control and RL approaches. Three dynamics systems (cart-pole, 1D and 2D quadrotor) and two control tasks (stabilization and trajectory tracking) are included. This simulation environment supports model-based and data-based approaches, expresses safety constraints, and captures real-world properties (such as uncertainty in physical properties and state estimation).

PyFlyt [45] is an open-source simulator using the PyBullet physics engine [44]. For RL research, *PyFlyt* includes compatibility with *Gymnasium* [46] and *PettingZoo* [47] (for multi-agent RL). Basic UAV components are expressed modularly to allow for flexible construction of UAVs, including support for multirotors and fixed-wings.

Isaac Gym (Aerial Gym) [28] ([29]), see Sec. II-A.

PRL4AirSim [37], see Sec. II-A.

Flightmare [38], see Sec. II-B.

gym-pybullet-drones [48], see Sec. II-E.

QuadSwarm [49], see Sec. II-E.

fixed-wing-gym [50], see Sec. II-D.

QPlane [51], see Sec. II-G.

D. Dynamics-focused simulators

RotorTM [52] is an open-source simulator for aerial object manipulation. Notably, this simulator considers cable-suspended loads and passive connection mechanisms between multiple vehicles, a feature lacking in other common simulators. In *RotorTM*, the cables are considered massless and connected to the robot's CoM. They can transition from taut to slack during task execution, allowing users to customize the number of robots and the type of payload (e.g. rigid body or point mass). Additionally, the simulator accommodates scenarios where aerial robots are rigidly attached to the load. *RotorTM* assumes negligible drag on the payload and aerial robot and disregards aerodynamic effects, considering rotor dynamics to be significantly faster than other factors.

MATLAB UAV Toolbox [53] is a general purpose toolbox for designing, simulating, testing, and deploying UAVs within MATLAB. It includes tools for algorithm development, flight log analysis, and simulation. The simulation capabilities include a cuboid simulation for quickly constructing new scenarios and a photorealistic 3D simulation environment with synthesized camera and LiDAR readings. The toolbox includes an interface for deploying directly to hardware through PX4-based autopilots. Additionally, the MAVLink protocol is supported. Researchers have explored using the *MATLAB UAV Toolbox* with flight simulators [54] such as *X-Plane*, *FlightGear*, and, in other works (as mentioned in [54]) *RealFlight*.

PyFly [50] is an open-source Python simulator designed for fixed-wing aircraft. It includes a 6-DoF aerodynamic model, wind effects, and stochastic turbulence. *fixed-wing-gym* [50] is an OpenAI Gym wrapper specifically tailored for *PyFly*, aiming at facilitating RL applications.

ARCAD [55], or *AirLab Rapid Controller and Aircraft Design*, is an open-source MATLAB simulator for fully-actuated

multirotors. Its primary goals are to expedite the modeling, design, and analysis of new aircraft and controllers and the visualization of tasks involving physical interactions, including controlled force-based tasks like writing text on a wall.

HIL-airmanip [56] offers a distinctive environment for simulating physical interactions between humans and aerial robots, enabling real-time human involvement. In this simulator, the forces exchanged between the human operator and a haptic interface are accurately measured and then transmitted to an aerial manipulator, which is modeled within the *RotorS* environment. This robotic system consists of a quadrotor combined with a 6-DoF arm mounted beneath it.

RotorPy [57] is an open-source Python simulator meant to be lightweight and focused on providing a comprehensive quadrotor model. Its development emphasizes accessibility, transparency, and educational value, initially created as a teaching tool for a robotics course at the University of Pennsylvania. In [57], the simulator's quadrotor model is extensively detailed, including 6-DoF dynamics, aerodynamic wrenches, actuator dynamics, sensors, and wind models. The model's validity is verified using a Crazyflie performing agile maneuvers. Additionally, the simulator includes a *Gymnasium* environment for RL applications.

RflySim [58] is a model-based design toolchain in MATLAB that can be used for SITL and HITL testing with an emphasis on model credibility and extensibility. Unreal is used for visual scene rendering, PX4 compatibility is built in, and multiple UAV models are supported.

Agilicious [59], see Sec. II-F.

E. Swarming simulators

gym-pybullet-drones [48] is an open-source environment designed for simulating multiple quadrotors with PyBullet [44] physics, tailored for research that combines control theory and ML. This library has interfaces for multi-agent and vision-based RL applications, utilizing the *Gymnasium* APIs [46]. It supports the definition of various learning tasks on a Crazyflie platform. Notably, *gym-pybullet-drones* includes realistic collisions and aerodynamic effects (e.g. drag, ground effect, and downwash). It includes example RL workflows for single agent and multi-agent scenarios, leveraging *Stable-baselines3* [60].

QuadSwarm [49] is an open-source Python library for multi-quadrotor simulation in RL applications, emphasizing fast simulation and the transfer of policies from simulation to the real-world. *QuadSwarm* provides diverse training scenarios and domain randomization to support RL applications, showcasing zero-shot transfer of RL control policies for single and multi-quadrotor scenarios. The physics model is based on the Crazyflie platform, with OpenGL used for rendering.

Potato [61] is based on data-oriented programming for large-scale swarm simulations. Like *Isaac Gym*, *Potato* relies on GPU computation rather than CPU. It includes basic dynamics for fixed-wing drones, quadrotors, and cars. *Potato* is not currently open-source, but the authors expressed the intention to open-source the quadrotor piece in the future.

F. Simulators part of flight stacks

Agilicious [59] contains a hardware description for a quadrotor with a Jetson TX2 and a software library specifically meant for autonomous and agile quadrotor flight. For simulation, it has a custom modular simulator that incorporates highly accurate aerodynamics based on blade-element momentum theory or with other tools like RotorS, HITL setups, and rendering engines such as Flightmare. The stack uses a custom license, but is free to use for academics after registration.

MRS UAV System [62] is a flight stack designed for replicable research through realistic simulations and real-world experiments. Its software stack includes a simulation environment built on Gazebo Classic, CoppeliaSim, or their MRS-multirotor-simulator for quadrotor dynamics, complete with realistic sensors and models. A key feature is its ongoing active use and maintenance. Moreover, this stack is frequently used for teams of multirotors.

CrazyChoir [63] is an open-source modular ROS 2 framework designed for conducting realistic simulations and experiments involving cooperating Crazyflie drones. For simulation, it builds on Webots with a SITL of the Crazyflie firmware.

Crazyswarm2 [64] is an open-source framework designed for controlling large indoor quadrotor swarms, specifically utilizing Crazyflie drones, similarly to CrazyChoir. For simulation it also relies on SITL of the firmware with a modular simulation framework that currently supports pure visualization or an *ad hoc* Python-based physics simulation.

Aerostack2 [65] is a versatile open-source flight stack designed to be compatible with various UAV platforms, including PX4, ArduPilot, DJI, and Crazyflie. For simulation purposes, Aerostack2 utilizes Gazebo with custom sensors. However, it does not currently have support for (S/H)ITL simulations.

CrazySim [66] is a SITL simulation pipeline for swarming research with the Crazyflie quadrotor. An individual instance of the Crazyflie flight stack is run for each UAV, with sensors and communication simulated in Gazebo, in order to test firmware code in simulation. Additionally, radio communication between the UAVs and ground station, including delays, are simulated. *sim_cf2* [67] is very similar to CrazySim for SITL simulation, but instead uses Gazebo Classic. Additionally, *sim_cf2* is more integrated with ROS 2 versus CrazySim offers a Gazebo plugin without ROS 2 dependencies.

G. Flight simulators

X-Plane [68] is a commercial cross-platform flight simulator. As with most flight simulators, the primary audience is pilots. X-Plane emphasizes realistic dynamics (focused on large-scale fixed-wing and helicopter vehicles) and includes simulated weather, wind, and lighting conditions. *X-PlaneROS* [69] is a X-Plane ROS 1 wrapper for controlling large-scale fixed-wing vehicles, extracting aircraft data from the simulator, and enables human-robot interaction. *QPlane* [51] is a RL toolkit for fixed-wing simulation that can use external flight simulators, such as X-Plane and/or FlightGear.

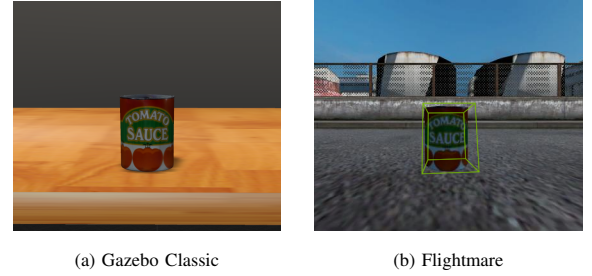


Figure 3. Toy can in Gazebo Classic and Flightmare simulation environments. In (b), the green bounding box denotes a detection by Deep Object Pose Estimation (DOPE) [74]. The comparable image in the Gazebo Classic simulation did not yield detections by DOPE due to its lower visual fidelity. Gazebo Classic utilizes OGRE for rendering, while Flightmare uses Unity.

FlightGear [70] is an open-source, user supported, cross-platform flight simulator. Some researchers have explored using this flight simulator for UAV simulation, such as in [71].

RealFlight [72] is a commercial Windows Radio Controlled (RC) flight simulator that includes small multirotor and fixed-wing vehicles. Researchers have adopted this flight simulator for UAV simulation, such as in [73].

III. UAV SIMULATOR COMPARISON

In Table IV, we summarize prominent positive and negative decision factors for all included simulators. We then provide three more detailed tables, for a subset of the simulators, that compare essential features of aerial simulators, using the selection criteria discussed in Table II. We exclude simulation speed as this varies depending on the environment’s complexity, physics model, rendering quality, and computer hardware. We encourage the reader to preform this test (as needed) for their application space. The detailed tables include extensively utilized simulators for aerial vehicles. For brevity, we omitted simulators that are less versatile or relatively new, leading to limited adoption.

Table V presents a comparison of notable features within the simulation environments. Different applications may require different physics fidelity, such as for environmental contact or physical accuracy. There is often a trade-off between speed and accuracy. We include the physics engines used in each simulator in Table V and refer the reader to [31] as needed for a comprehensive analysis of different physics engines. We denote the rendering capabilities of the simulators. Notably, OpenGL and OGRE rendering are often regarded as having low visual fidelity, while Vulkan, Unity, and Unreal rendering are considered to offer high visual fidelity. Figure 3 depicts a comparison of camera views in an OGRE environment versus a Unity environment. The Operating System (OS) subcategories are Linux, Windows, and Mac, denoted as “L,” “W,” “M,” respectively. “RL” is included as an interface to indicate specialization for RL applications. The category denoted as (S/H)ITL includes interfaces for both SITL and HITL capabilities, with “CF” representing Crazyflie. Finally, we denote the license and whether each simulator is open-source and refer the reader to [75] for more information on various license types. Lastly, we include a column “Active”

Table IV
PROMINENT DECISION FACTORS TO CONSIDER FOR EACH SIMULATOR

Simulator	Positive Factors	Negative Factors	Ref.
Gazebo Classic	Built into ROS 1, large community, diverse platform support	Low visual fidelity, slow comp., phasing out	[19]
RotorS (with Gazebo Classic)	Key UAV modeling and algorithms for Gazebo Classic	Built on Gazebo Classic (default in Gazebo)	[22]
CrazyS (with Gazebo Classic)	Crazyflie specific extension for Gazebo Classic	Built on Gazebo Classic, limited-perception	[23]
PX4 SITL Gazebo Classic	PX4 SITL, large community, allows multi-sensor config.	Built on Gazebo Classic, PX4 exclusive	[24]
Gazebo	Built into ROS 2, dynamic environment loading	Low visual fidelity	[25]
Isaac Sim	Photorealistic, high-fidelity, NVIDIA supported	Not open-source	[26]
Pegasus (with Isaac Sim)	Extended multirotor dynamics and sensors, PX4 integration	Back end (Isaac Sim) is not open-source	[27]
Isaac Gym	GPU-accelerated RL simulations, NVIDIA supported	Not open-source	[28]
Aerial Gym (with Isaac Gym)	Highly parallelizable, GPU-enabled geometric controllers	Back end (Isaac Gym) is not open-source	[29]
Webots	User-friendly, diverse platform support	Focused on ground robots, basic aerodynamics	[30]
CoppeliaSim	Large number of physics engines and platforms	Not fully open-source	[33]
AirSim	Photorealistic, large sensor and weather support	Not actively maintained, substantial compute	[36]
PRL4AirSim (with AirSim)	Parallel training for RL using AirSim	Built on AirSim	[37]
Flightmare	Photorealistic, compute minimized, built for ML	Not actively maintained	[38]
FlightGoggles	Photorealistic, vehicle-in-the-loop simulation	Not actively maintained	[40]
FastSim	Photorealistic, extensive sensors, modular, generalized	Brand-new, code not released at time of writing	[41]
MARSIM	LiDAR measurement simulation, lightweight computation	Solely LiDAR focused	[42]
MuJoCo	Common for ML applications, parallel processing	Low visual fidelity	[35]
safe-control-gym	Safety-focused RL, benchmarks, control vs RL comparisons	Simplistic dynamical systems	[43]
PyFlyt	Gymnasium & PettingZoo integration, modular UAVs	Low visual fidelity	[45]
RotorTM	Cable-suspended loads and passive vehicle connections	Cables massless, negligible drag on payload	[52]
MATLAB UAV Toolbox	General toolbox for UAV design to deployment	Not open-source, UAV exclusive	[53]
PyFly	Fixed-wing control specific, includes wind and turbulence	No physical environment or rendering	[50]
fixed-wing-gym (with PyFly)	Gym wrapper for PyFly for RL applications	No physical environment or rendering	[50]
ARCAD	Fully-actuated UAV design, physical interaction tasks	Specific to fully-actuated UAVs	[55]
HIL-airmanip	Human-UAV interaction, haptic feedback	Specialized domain, RotorS backend	[56]
RotorPy	Comprehensive quadrotor model and aerodynamic effects	Simplistic environments	[57]
RflySim	Model-based design, SITL and HITL, Unreal, PX4	Separated into free, full, and enterprise versions	[58]
gym-pybullet-drones	Swarming, control and ML focused, aerodynamic effects	Low visual fidelity	[48]
QuadSwarm	Swarming, RL, sim-to-real transfer	Crazyflie physics model, low visual fidelity	[49]
Potato	Swarming, data-oriented programming, GPU computation	Not open-source	[61]
Agilicious	Agile flight, aerodynamics, open-hardware	Custom license, specialized platform	[59]
MRS UAV System	Onboard vehicle, integration with common simulators	Entire flight stack	[62]
CrazyChoir	ROS 2 framework, distributed swarm control, SITL	Crazyflie specific	[63]
Crazyswarm2	Large indoor swarms, onboard computation, high scalability	Crazyflie specific	[64]
Aerostack2	Compatibility with various autopilots, Gazebo-based	Does not support (S/H)ITL	[65]
CrazySim	Swarming SITL, simulated radios, Gazebo-based	Crazyflie specific	[66]
sim_cf2	Swarming SITL, integrated ROS 2 support	Crazyflie specific, built on Gazebo Classic	[67]
X-Plane	Flight simulator, weather, wind, lighting	Commercial, primarily for large-scale aircraft	[68]
X-PlaneROS (with X-Plane)	ROS 1 wrapper, fixed-wings, human-robot interaction	Primarily for large-scale aircraft	[69]
QPlane	RL toolkit for fixed-wings, uses external flight simulator	Primarily for large-scale aircraft	[51]
FlightGear	Open-source, user-supported flight simulator	Primarily for large-scale aircraft	[70]
RealFlight	RC flight simulator for small multirotors and fixed-wings	Commercial, Windows based	[72]

indicating the maintenance status at the time of writing this paper. Simulators under active maintenance are marked with ✓. Simulators that have been inactive but show some commits and responses to issues in the past two years are marked with *. Finally, simulators that are intentionally no longer maintained or have been inactive for more than two years are marked with ✗. As maintenance statuses may change over time, we advise readers to consider this information as a snapshot and to reevaluate before choosing a simulator.

Table VI compares the vehicle types that can be simulated, referencing the dynamics detailed in the “UAV Dynamics Background” sidebar. In the “Swarms” column, we specify packages designed for swarm purposes with ✓, packages that allow multiple vehicles (though not specifically designed for swarms) with *, and packages intended solely for single vehicles with ✗. We include columns for cars and other systems (e.g. manipulators, quadrupeds, humanoids) since researchers may require simulating interactions between different platforms with their intended UAV.

Table VII provides a comparison of supported sensors for each simulator. Segmentation, magnetometer, and barometer are abbreviated as “Seg,” “Mag,” and “Baro,” respectively.

We indicate features, vehicle types, and sensors supported in the base configurations of these simulators, acknowledging that many of them can be extended for additional support.

IV. DISCUSSION

The aerial robotics community has undeniably made significant strides in the development of simulators. However, a significant challenge lies in the fact that the specific requirements of various research groups tend to be platform-dependent and application-dependent, making it challenging to provide to all needs with a single simulator. Moreover, there is a growing consensus that exploring diverse solutions, rather than relying solely on one, can yield more favorable outcomes. Conversely, there is a compelling argument for standardization within this domain, as it would greatly facilitate benchmarking efforts and foster collaboration among researchers. Striking a balance between these two perspectives appears to be the most prudent approach. This entails directing resources and effort towards a select few simulators to harness the advantages inherent in both sides of the spectrum mentioned earlier. As a result, several key themes emerge, including the role of aerodynamics in simulation, the need for benchmarking, the relationship between academia and industry, data sharing, and the challenges associated with maintaining these simulators.

Aerodynamics and Simulation: One central topic revolves around the role of aerodynamics in UAV simulations. While it is acknowledged that many successful UAV applications do not necessitate intricate aerodynamic modeling, in some scenarios such modeling becomes indispensable. Particular attention is drawn to scenarios involving UAVs navigating in constrained environments, adapting to dynamic environmental conditions, or engaging in interactions with other drones. In these cases, there is a critical need for incorporating aerodynamics into

simulator development, which is even more valid for fixed-wing flight. On the other hand, it is important to note that for most of the applications involving multirotors, even for aerial manipulation tasks, aerodynamics play a less significant role.

Benchmarking and Standardization: The large number of simulators in existence necessitates benchmarking and standardization in this field. The absence of a unified benchmarking framework and standardization practices pose substantial challenges for researchers and developers alike. Addressing this issue emerges as a primary objective, as it has become evident that standardized benchmarking is paramount for enhancing the reproducibility and comparability of research within the UAV and more generally, the robotics domain, as many simulators are designed to be general purpose, such as the universal simulators from Sec. II-A.

Academia versus Industry: It is evident from the data reported in the tables that there is a distinction between simulators developed in academic and industrial settings. It is universally acknowledged that academic simulators have played a pivotal role in advancing research. However, academic simulators often grapple with sustainability challenges, particularly after the researchers responsible for their development, mainly doctoral students, graduate. In contrast, industry-backed simulators are characterized by robust support, continuous documentation, and sustained evolution. Striking a harmonious balance between academic and industry-driven simulator development emerges as a critical goal for the future.

Data Sharing and Collaboration: What also emerges from the above discussion is the need for data sharing within the research community and with industry partners. It is pivotal to emphasize the importance of sharing simulator data and models for making progress in the field. The potential advantage of shared datasets, particularly in the domains of perception and autonomous navigation, are evident. Collaborative endeavors and knowledge sharing among researchers can catalyze forces for simulator improvements and innovation.

Resource Identification: The numerous simulators discussed in this paper highlight the challenges newcomers face when selecting the most suitable simulator for their research. The demand for resources to streamline the selection process is clear. This guided the authors’ choice to write a survey to assist researchers and developers in navigating the diverse landscape of simulators and making well-informed choices.

Accessibility and Maintenance: When selecting a simulator, it is crucial to weigh factors like accessibility and maintainability. The simulator’s licensing has a significant impact on its utility and adaptability to research requirements. Simulators lacking open-source availability, a free proprietary license, or an academic license, can pose obstacles to replicating work. We encourage researchers to understand the license restrictions for a chosen simulator. Additionally, open-source simulators can be preferable when custom modifications to the simulator’s source code are necessary due to missing features. Furthermore, evaluating the long-term sustainability of a simulator is essential. A simulator that is no longer actively maintained may become unstable on newer operating systems

Table V
COMPARISON OF FEATURES FOR WIDELY USED UAV SIMULATORS: INCLUDED (✓), PARTIALLY INCLUDED (*), AND NOT INCLUDED (✗)

Simulator	Physics Engine	Rendering	OS			Interfaces	(S/H)ITL	License	Open-Source	Active	Ref.
			L	W	M						
Gazebo Classic (RotorS, CrazyS, PX4 SITL)	ODE, Bullet, DART, Simbody	OGRE	✓	* (✗, ✗, ✗, ✗)	✓ (✗, ✗, ✗, ✓)	ROS 1/2, C++, RL	PX4, ArduPilot, CF	Apache-2.0	✓	✓ (✗, *, ✓)	[19]–[24]
Gazebo	Bullet, DART, TPE	OGRE	✓	*	✓	ROS 1/2, C++, Python, RL	PX4, ArduPilot, CF	Apache-2.0	✓	✓	[25]
Isaac (Pegasus, Aerial Gym)	NVIDIA® PhysX, Flex	Vulkan	✓	✗	✗	ROS 1/2, Python, RL	Pegasus: PX4	Proprietary (BSD 3)	✗ (✓, ✓)	✓	[26]–[29]
Webots	ODE	OpenGL	✓	✓	✓	ROS 1/2, C/C++, Python, MATLAB, Java	ArduPilot, CF	Apache-2.0	✓	✓	[30]
CoppeliaSim	Bullet, ODE, Vortex, Newton, MuJoCo	OpenGL	✓	✓	✓	ROS 1/2, C/C++, Python, MATLAB, Java, Lua, Octave	—	GNU GPL, Commercial	*	✓	[33]
AirSim	NVIDIA® PhysX	Unreal, Unity	✓	✓	✓	ROS 1, C++, Python, C#, Java, RL	PX4, ArduPilot	MIT	✓	✗	[36]
Flightmare	Ad hoc, Gazebo Classic	Unity	✓	✗	✗	ROS 1, C++, RL	—	MIT	✓	✗	[38]
FlightGoggles	Ad hoc	Unity	✓	*	✗	ROS 1, C++	Motion Capture	MIT	✓	✗	[40]
gym-pybullet-drones	PyBullet	OpenGL	✓	*	✓	Python, RL	Betaflight, CF	MIT	✓	✓	[48]
RotorTM	Ad hoc	OpenGL	✓	✗	✗	ROS 1, Python, MATLAB	—	GNU GPL	✓	✓	[52]
MATLAB UAV Toolbox	MATLAB	Unreal	✓	✓	✓	ROS 2, MATLAB	PX4	Proprietary, Commercial	✗	✓	[53]

Table VI
COMPARISON OF VEHICLE TYPES FOR WIDELY USED UAV SIMULATORS: INCLUDED (✓), PARTIALLY INCLUDED (*), AND NOT INCLUDED (✗)

Simulator	Multirotors			Helicopters	Fixed-wings	Aerial Manip.	Swarms	Cars	Other Systems	Ref.
	Basic	Drag	Wind							
Gazebo (Classic & New)	✓	✓	✓	✗	✓	*	*	✓	✓	[19], [25]
Isaac (Pegasus, Aerial Gym)	✓	✗ (✓, ✗)	✗	✓	✗	✗	✓	✓ (✗, ✗)	✓ (✗, ✗)	[26]–[29]
Webots	✓	✗	✗	✓	✗	✗	*	✓	✓	[30]
CoppeliaSim	✓	✓	*	✓	✗	*	*	✓	✓	[33]
AirSim	✓	✓	✓	✗	✗	✗	*	✓	✗	[36]
Flightmare	✓	✓	✗	✗	✗	✗	✓	✗	✗	[38]
FlightGoggles	✓	✓	✗	✗	✗	✗	✗	✓	✗	[40]
gym-pybullet-drones	✓	✓	✗	✗	✗	✗	✓	✗	✗	[48]
RotorTM	✓	✗	✗	✗	✗	✓	✓	✗	✗	[52]
MATLAB UAV Toolbox	✓	✓	✓	✗	✓	✗	*	✗	✗	[53]

and may lack updated integration support for middleware like ROS. This can cause researchers to divert valuable time and effort from their experiments to maintaining the simulator.

V. CONCLUSIONS

Selecting a simulator that is best for a particular application space can be challenging, but rewarding when it increases safety and reduces testing time and cost. In this paper, we discussed some of the prominent robotic simulators for aerial vehicles. We enumerate possible decision factors to consider when selecting a simulator and we compare features, included vehicle types, and integrated sensors across many widely used

simulation packages. For researchers new to the field, we recommend starting with a well-supported universal simulator (e.g. Gazebo) and then using this paper to identify specialized solutions as needed. We hope that this analysis will be valuable to the community when embarking on aerial vehicle research and selecting a simulation environment.

ACKNOWLEDGMENT

We gratefully acknowledge feedback from Geoffrey Biggs, Addisu Taddese, Jaeyoung Lim, Jay Patrikar, Marcelo Jacinto, João Pinto, Mihir Kulkarni, Marc Freese, Yunlong Song, Jacopo Panerati, Angela Schoellig, and Guanrui Li.

Table VII
COMPARISON OF INCLUDED SENSORS FOR WIDELY USED AERIAL VEHICLE SIMULATORS: INCLUDED (✓) AND NOT INCLUDED (✗)

Simulator	RGB	Depth	Seg.	Point Cloud	IMU	Mag.	GPS	Baro.	LiDAR	Optical Flow	Ref.
Gazebo (Classic & New)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	[19], [25]
Isaac Sim (Pegasus)	✓	✓	✓	✓	✓	✗ (✓)	✗ (✓)	✗ (✓)	✓	✓	[26], [27]
Isaac Gym (Aerial Gym)	✓	✓	✓	✗	✗	✗	✗	✗	✗	✓	[28], [29]
Webots	✓	✓	✗	✗	✓	✓	✓	✗	✓	✗	[30]
CoppeliaSim	✓	✓	✗	✓	✓	✗	✓	✗	✓	✗	[33]
AirSim	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	[36]
Flightmare	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	[38]
FlightGoggles	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓	[40]
gym-pybullet-drones	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	[48]
RotorTM	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	[52]
MATLAB UAV Toolbox	✓	✓	✓	✓	✓	✗	✓	✗	✓	✗	[53]

REFERENCES

- [1] S. Leutenegger, *et al.*, *Flying Robots*. Springer Handbook of Robotics, Springer, 2016, pp. 623–670.
- [2] A. Ollero, *et al.*, “Past, Present, and Future of Aerial Robotic Manipulators,” *IEEE Trans. Rob.*, vol. 38, no. 1, pp. 626–645, 2022.
- [3] A. Kolling, *et al.*, “Human Interaction With Robot Swarms: A Survey,” *IEEE Trans. on Hum.-Mach. Syst.*, vol. 46, no. 1, pp. 9–26, 2016.
- [4] C. K. Liu *et al.*, “The Role of Physics-Based Simulators in Robotics,” *Ann. Rev. of Contr., Rob., and Aut. Syst.*, vol. 4, no. 1, pp. 35–58, 2021.
- [5] J. Collins, *et al.*, “A Review of Physics Simulators for Robotic Applications,” *IEEE Acc.*, vol. 9, pp. 51 416–51 431, 2021.
- [6] J. Saunders, *et al.*, “Autonomous aerial robotics for package delivery: A technical review,” *J. of Fie. Rob.*, pp. 1–47, 2023.
- [7] A. Mairaj, *et al.*, “Application specific drone simulators: Recent advances and challenges,” *Sim. Mod. Pr. and Th.*, vol. 94, pp. 100–117, 2019.
- [8] M. Faessler, *et al.*, “Differential Flatness of Quadrotor Dynamics Subject to Rotor Drag for Accurate Tracking of High-Speed Trajectories,” *Robot. Autom. Lett.*, vol. 3, no. 2, pp. 620–626, 2018.
- [9] G. Shi, *et al.*, “Neural-Swarm2: Planning and Control of Heterogeneous Multirotor Swarms Using Learned Interactions,” *IEEE Trans. Rob.*, vol. 38, no. 2, pp. 1063–1079, 2022.
- [10] M. Basescu *et al.*, “Direct NMPC for Post-Stall Motion Planning with Fixed-Wing UAVs,” in *Int. Conf. Rob. Aut.*, 2020, pp. 9592–9598.
- [11] F. Sobolic *et al.*, “Nonlinear Agile Control Test Bed for a Fixed Wing Aircraft in a Constrained Environment,” in *AIAA Inf. Aer. Conf. and AIAA Unm.... Unl. Conf.*, 2009, p. 1927.
- [12] S. Hoerner, “Fluid-dynamic lift,” *Hoerner Fluid Dynamics*, 1985.
- [13] M. Basescu, *et al.*, “Precision Post-Stall Landing Using NMPC With Learned Aerodynamics,” *Robot. Autom. Lett.*, vol. 8, no. 5, pp. 3031–3038, 2023.
- [14] M. Basescu, *et al.*, “Agile fixed-wing UAVs for urban swarm operations,” *Fie. Rob.*, vol. 3, pp. 725–765, 2023.
- [15] P. R. Ambati *et al.*, “Robust auto-landing of fixed-wing UAVs using neuro-adaptive design,” *Contr. Eng. Pr.*, vol. 60, pp. 218–232, 2017.
- [16] A. Gomez-Tamm, *et al.*, “Current State and Trends on Bioinspired Actuators for Aerial Manipulation,” in *Int. Work. on Res., Ed. and Dev. of Unm. Aer. Sys.*, 2019, pp. 352–361.
- [17] X. Meng, *et al.*, “Survey on Aerial Manipulator: System, Modeling, and Control,” *Robotica*, vol. 38, no. 7, pp. 1288–1317, 2020.
- [18] C. Gabellieri, *et al.*, “Equilibria, Stability, and Sensitivity for the Aerial Suspended Beam Robotic System Subject to Parameter Uncertainty,” *IEEE Trans. Rob.*, vol. 39, no. 5, pp. 3977–3993, 2023.
- [19] N. Koenig *et al.*, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Int. Conf. Int. Rob. Syst.*, vol. 3, 2004, pp. 2149–2154 vol.3.
- [20] A. Suarez, *et al.*, “Compliant Bimanual Aerial Manipulation: Standard and Long Reach Configurations,” *IEEE Acc.*, vol. 8, pp. 88 844–88 865, 2020.
- [21] C. A. Dimmig, *et al.*, “A Small Form Factor Aerial Research Vehicle for Pick-and-Place Tasks with Onboard Real-Time Object Detection and Visual Odometry,” in *Int. Conf. Int. Rob. Syst.*, 2023, pp. 6289–6296.
- [22] F. Furrer, *et al.*, “RotorS — A Modular Gazebo MAV Simulator Framework,” *ROS The Complete Reference*, vol. 1, pp. 595–625, 2016.
- [23] G. Silano, *et al.*, “CrazyS: A Software-In-The-Loop Platform for the Crazyflie 2.0 Nano-Quadcopter,” in *Med. Conf. on Contr. and Aut.*, 2018, pp. 352–357.
- [24] PX4, “Gazebo Classic Simulation.” [Online]. Available: http://docs.px4.io/main/en/sim_gazebo_classic/
- [25] Open Robotics, “Gazebo.” [Online]. Available: <https://gazebo.org/>
- [26] NVIDIA, “Isaac Sim.” [Online]. Available: <https://developer.nvidia.com/isaac-sim>
- [27] M. Jacinto, *et al.*, “Pegasus Simulator: An Isaac Sim Framework for Multiple Aerial Vehicles Simulation,” in *Int. Conf. on Unm. Air. Sys.* IEEE, 2024, pp. 917–922.
- [28] V. Makoviychuk, *et al.*, “Isaac Gym: High performance GPU-based physics simulation for robot learning,” *arXiv:2108.10470*, 2021.
- [29] M. Kulkarni, *et al.*, “Aerial Gym–Isaac Gym Simulator for Aerial Robots,” *arXiv:2305.16510*, 2023.
- [30] O. Michel, “Cyberbotics Ltd. Webots™: Professional Mobile Robot Simulation,” *Int. J. of Adv. Rob. Sys.*, vol. 1, no. 1, p. 5, 2004.
- [31] T. Erez, *et al.*, “Simulation tools for model-based robotics: Comparison of Bullet, Havok, MuJoCo, ODE and PhysX,” in *Int. Conf. Rob. Aut.*, 2015, pp. 4397–4404.
- [32] X. Gu, *et al.*, “Design and Dynamics Simulation of a Triphibious Robot in Webots Environment,” in *Int. Conf. on Mech. and Aut.*, 2021, pp. 1268–1273.
- [33] E. Rohmer, *et al.*, “V-REP: A versatile and scalable robot simulation framework,” in *Int. Conf. Int. Rob. Syst.*, 2013, pp. 1321–1326.
- [34] P. Udvardy, *et al.*, “Simulation of obstacle avoidance of an UAV,” in *New Tr. in Av.Dev.*, 2020, pp. 245–249.
- [35] E. Todorov, *et al.*, “MuJoCo: A physics engine for model-based control,” in *Int. Conf. Int. Rob. Syst.*, 2012, pp. 5026–5033.
- [36] S. Shah, *et al.*, “AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles,” in *Fie. and Ser. Rob.*, 2018, pp. 621–635.
- [37] J. Saunders, *et al.*, “Parallel Reinforcement Learning Simulation for Visual Quadrotor Navigation,” in *Int. Conf. Rob. Aut.*, 2023, pp. 1357–1363.
- [38] Y. Song, *et al.*, “Flightmare: A Flexible Quadrotor Simulator,” in *Conf. Robot Learning*. PMLR, 2021, pp. 1147–1157.
- [39] Y. Song, *et al.*, “Reaching the limit in autonomous racing: Optimal control versus reinforcement learning,” *Sci. Rob.*, vol. 8, no. 82, 2023.
- [40] W. Guerra, *et al.*, “FlightGoggles: Photorealistic Sensor Simulation for Perception-driven Robotics using Photogrammetry and Virtual Reality,” in *Int. Conf. Int. Rob. Syst.*, 2019, pp. 6941–6948.
- [41] C. Cui, *et al.*, “FastSim: A Modular and Plug-and-Play Simulator for Aerial Robots,” *Robot. Autom. Lett.*, pp. 1–8, 2024.
- [42] F. Kong, *et al.*, “MARSIM: A Light-Weight Point-Realistic Simulator for LiDAR-Based UAVs,” *Robot. Autom. Lett.*, vol. 8, no. 5, pp. 2954–2961, 2023.
- [43] Z. Yuan, *et al.*, “Safe-Control-Gym: A Unified Benchmark Suite for Safe Learning-Based Control and Reinforcement Learning in Robotics,” *Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11 142–11 149, 2022.
- [44] E. Coumans *et al.*, “PyBullet, a python module for physics simulation for games, robotics and machine learning,” 2016. [Online]. Available: <https://pybullet.org/>

- [45] J. J. Tai, *et al.*, “PyFlyt—UAV Simulation Environments for Reinforcement Learning Research,” *arXiv:2304.01305*, 2023.
- [46] M. Towers, *et al.*, “Gymnasium,” Mar. 2023. [Online]. Available: <https://zenodo.org/record/8127025>
- [47] J. Terry, *et al.*, “PettingZoo: Gym for Multi-Agent Reinforcement Learning,” in *Neural Inf. Process. Syst.*, M. Ranzato, *et al.*, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 15 032–15 043.
- [48] J. Panerati, *et al.*, “Learning to Fly—a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control,” in *Int. Conf. Int. Rob. Syst.*, 2021, pp. 7512–7519.
- [49] Z. Huang, *et al.*, “QuadSwarm: A Modular Multi-Quadrotor Simulator for Deep Reinforcement Learning with Direct Thrust Control,” *arXiv:2306.09537*, 2023.
- [50] E. Bøhn, *et al.*, “Deep Reinforcement Learning Attitude Control of Fixed-Wing UAVs Using Proximal Policy optimization,” in *Int. Conf. on Unm. Air. Sys.*, 2019, pp. 523–533.
- [51] D. J. Richter *et al.*, “QPlane: An Open-Source Reinforcement Learning Toolkit for Autonomous Fixed Wing Aircraft Simulation,” in *ACM Mult. Syst. Conf.*, ser. MMSys ’21, 2021, pp. 261–266.
- [52] G. Li, *et al.*, “RotorTM: A Flexible Simulator for Aerial Transportation and Manipulation,” *IEEE Trans. Rob.*, pp. 1–20, 2023.
- [53] MATLAB, “UAV Toolbox.” [Online]. Available: <https://www.mathworks.com/products/uav.html>
- [54] N. Horri *et al.*, “A Tutorial and Review on Flight Control Co-Simulation Using Matlab/Simulink and Flight Simulators,” *Automation*, vol. 3, no. 3, pp. 486–510, 2022.
- [55] A. Keipour, *et al.*, “UAS Simulator for Modeling, Analysis and Control in Free Flight and Physical Interaction,” in *AIAA SCI. For.*, 2023, p. 1279.
- [56] E. Cuniato, *et al.*, “A hardware-in-the-loop simulator for physical human-aerial manipulator cooperation,” in *Int. Conf. on Adv. Rob.*, 2021, pp. 830–835.
- [57] S. Folk, *et al.*, “RotorPy: A Python-based Multicopter Simulator with Aerodynamics for Education and Research,” *arXiv:2306.04485*, 2023.
- [58] X. Dai, *et al.*, “RFlySim: Automatic test platform for UAV autopilot systems with FPGA-based hardware-in-the-loop simulations,” *Aerospace Science and Technology*, vol. 114, p. 106727, 2021.
- [59] P. Foehn, *et al.*, “Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight,” *Sci. Rob.*, vol. 7, no. 67, 2022.
- [60] A. Raffin, *et al.*, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *J. of Mach. Lear. Res.*, vol. 22, no. 1, 2021.
- [61] J. Li, *et al.*, “Potato: A Data-Oriented Programming 3D Simulator for Large-Scale Heterogeneous Swarm Robotics,” *arXiv:2308.12698*, 2023.
- [62] T. Baca, *et al.*, “The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles,” *J. of Int. & Rob. Sys.*, vol. 102, no. 1, p. 26, 2021.
- [63] L. Pichierri, *et al.*, “CrazyChoir: Flying Swarms of Crazyflie Quadrotors in ROS 2,” *Robot. Autom. Lett.*, vol. 8, no. 8, pp. 4713–4720, 2023.
- [64] J. A. Preiss, *et al.*, “Crazyswarm: A large nano-quadcopter swarm,” in *Int. Conf. Rob. Aut.*, 2017, pp. 3299–3304.
- [65] M. Fernandez-Cortizas, *et al.*, “Aerostack2: A Software Framework for Developing Multi-robot Aerial Systems,” *arXiv:2303.18237*, 2023.
- [66] C. Llanes, *et al.*, “CrazySim: A Software-in-the-Loop Simulator for the Crazyflie Nano Quadrotor,” in *Int. Conf. Rob. Aut.*, 2024.
- [67] “sim_cf2.” [Online]. Available: https://github.com/CrazyflieTHI/sim_cf2
- [68] “X-Plane.” [Online]. Available: <https://www.x-plane.com/>
- [69] I. Navarro, *et al.*, “SoRTS: Learned Tree Search for Long Horizon Social Robot Navigation,” *Robot. Autom. Lett.*, vol. 9, no. 4, pp. 3759–3766, 2024.
- [70] A. R. Perry, “The FlightGear Flight Simulator,” in *USENIX An. Tech. Conf.*, vol. 686, 2004, pp. 1–12.
- [71] Y. A. Prabowo, *et al.*, “Hardware in-the-loop simulation for visual servoing of fixed wing UAV,” in *Int. Conf. on El. Eng. and Infor.*, 2015, pp. 247–252.
- [72] “RealFlight.” [Online]. Available: <https://www.realflight.com/>
- [73] S. J. Carlson *et al.*, “The MiniHawk-VTOL: Design, Modeling, and Experiments of a Rapidly-prototyped Tiltrotor UAV,” in *Int. Conf. on Unm. Air. Sys.*, 2021, pp. 777–786.
- [74] J. Tremblay, *et al.*, “Deep Object Pose Estimation for Semantic Robotic Grasping of Household Objects,” in *Conf. Robot Learning*, 2018.
- [75] M. Ballhausen, “Free and open source software licenses explained,” *Computer*, vol. 52, no. 6, pp. 82–86, 2019.